

Financial Reporting System

A secure full-stack application for managing accounts, transactions, and financial summaries

Angular 17 • Spring Boot 3 • PostgreSQL • JWT Authentication

Agenda

- 01** What the application does
- 02** Technology stack
- 03** Core functionality: Accounts, Transactions, Reporting
- 04** Frontend architecture (Angular)
- 05** Backend architecture (Spring Boot)
- 06** JWT authentication flow
- 07** Security filter chain
- 08** System architecture overview

What the Application Does

A web app where users securely track their personal or business finances.



Secure Login

Register and log in with JWT-based authentication



Manage Accounts

Create financial accounts across 5 standard types



Record Transactions

Track money in/out — balances update automatically



View Reports

Live dashboard summarizing overall financial position

Technology Stack

Layer	Technology	Notes
Frontend	Angular 17	Standalone components, Reactive Forms
Backend	Spring Boot 3 (Java 17)	Layered REST API architecture
Database	PostgreSQL	Managed via Flyway migrations
Security	JWT (JSON Web Tokens)	Stateless authentication
Communication	REST over HTTP	Frontend never touches the DB directly

Core Domain: Accounts & Transactions

Accounts

A financial “bucket” — e.g. Checking, Credit Card, Salary

-
- Five standard types: Asset, Liability, Equity, Revenue, Expense
-
- Full CRUD: create, view, edit, delete
-
- Each account is owned by exactly one user

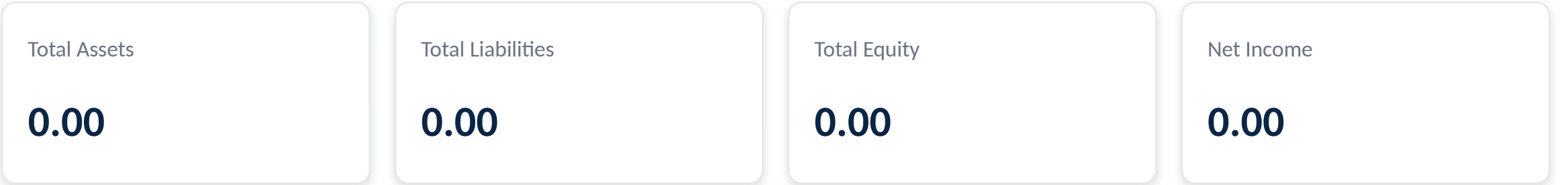
Transactions

A single movement of money: Debit or Credit

-
- Linked to one account, with amount, date, category
-
- Account balance updates automatically on create, edit, or delete
-
- No manual balance recalculation needed

Reporting Dashboard

Aggregates every account and transaction into one live financial snapshot.



Balance by Account Type

ASSET	0.00
LIABILITY	0.00
EQUITY	0.00
REVENUE	0.00
EXPENSE	0.00

Why this matters

This turns raw transaction data into a readable financial picture — the true “reporting” part of the system — without the user doing any manual math.

Frontend Architecture (Angular 17)

core/services

One API-calling service per resource (auth, accounts, transactions, reports)

core/guards

Blocks access to protected pages if not logged in

core/interceptors

Auto-attaches JWT token to every outgoing request

auth / accounts / transactions

Feature pages: login, register, list & form components

dashboard

Financial summary page

shared/navbar

Persistent navigation shown once logged in

Frontend: Key Concepts

Reactive Forms

Validation rules defined in TypeScript — real-time feedback before submit

Route Guards

Direct URL access to /dashboard redirects to /login if not authenticated

HTTP Interceptor

Every request auto-attaches the JWT; 401 responses trigger auto-logout

Lazy Loading

Each page's code downloads only when the user navigates to it

Backend Architecture (Spring Boot)

Controller

Receives HTTP requests, maps them to endpoints (e.g. AccountController)

Service

Business logic — e.g. auto-updating balances when a transaction changes

Repository

Data access via Spring Data JPA — talks to PostgreSQL

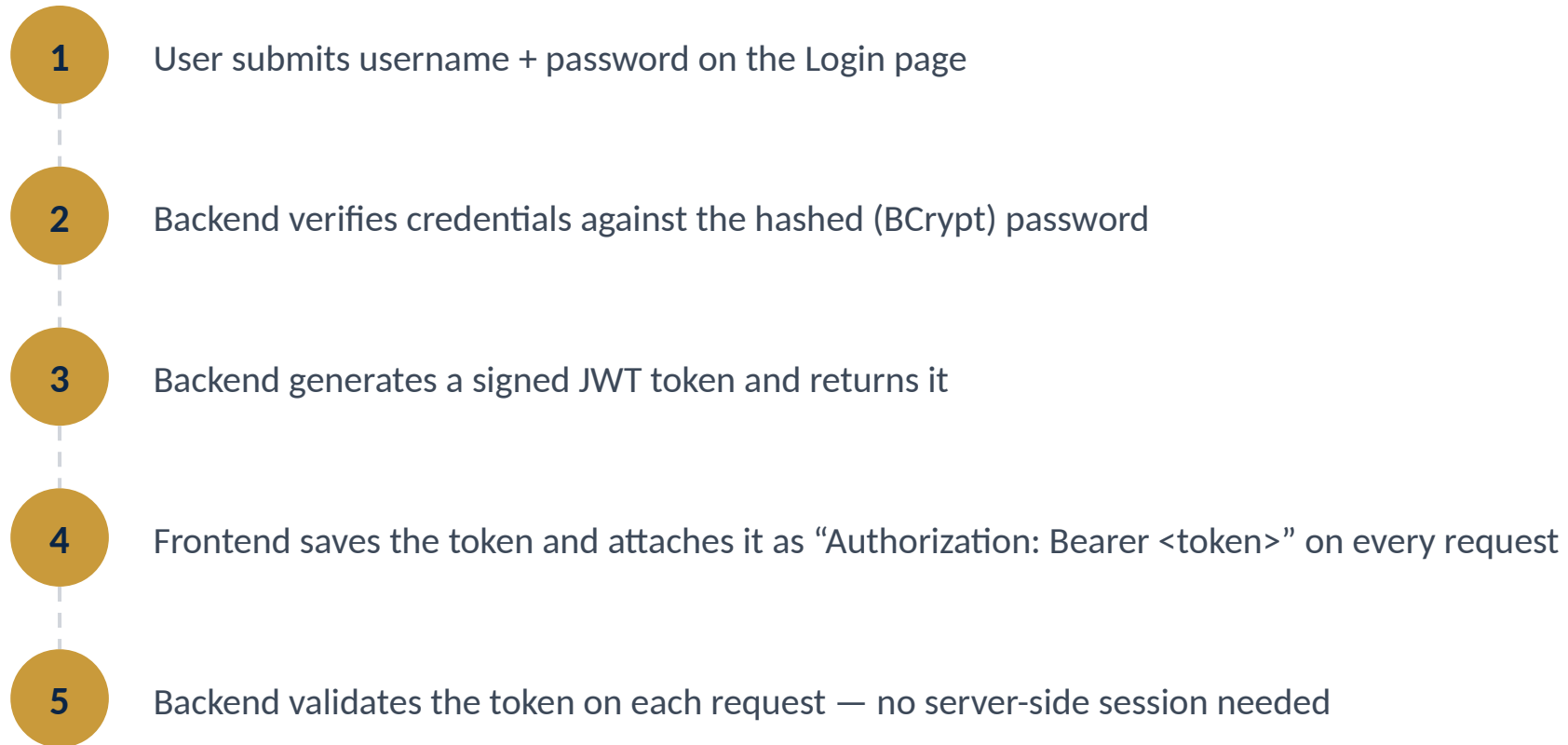
Security

JwtUtil, JwtAuthFilter, SecurityConfig — authentication & authorization

DTO / Exception

Clean request/response shapes and centralized error handling

JWT Authentication Flow



Security Filter Chain

Every request passes through a chain of checks before reaching a controller.

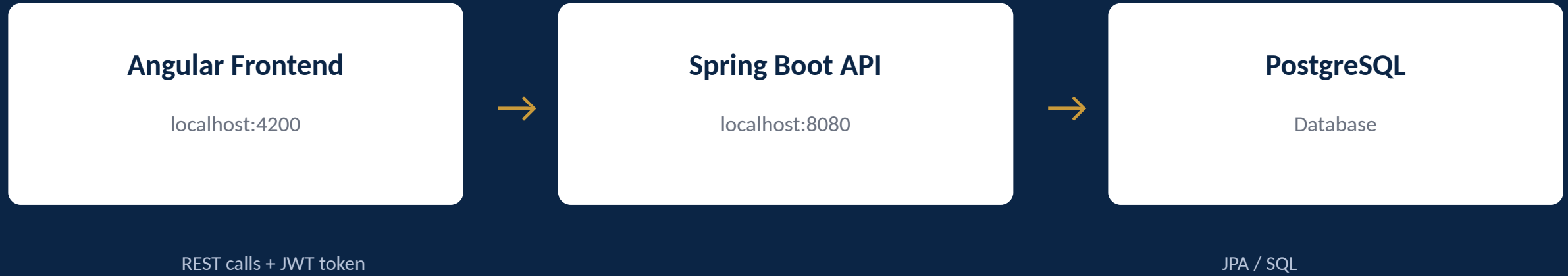


What JwtAuthFilter does on every request:

Reads the “Authorization: Bearer <token>” header

-
- Extracts and validates the JWT (signature + expiry)
-
- If valid, tells Spring Security who the authenticated user is
-
- Runs before Spring's own login filter — added via `addFilterBefore()`

System Architecture



The frontend never accesses the database directly — every request goes through the authenticated REST API.

Thank You

Questions & Discussion
